

使用 BigQuery 和 Google 地圖的 MCP 伺服器，建構 Location Intelligence ADK 代理

來源：<https://codelabs.developers.google.com/adk-mcp-bigquery-maps?hl=zh-tw>

步驟數：9

在本程式碼實驗室中，您將建構 ADK 代理，並從 BigQuery 和 Google 地圖的 MCP 伺服器取得工具。代理程式會協調要求，在洛杉磯尋找適合開設高檔麵包店的地點。

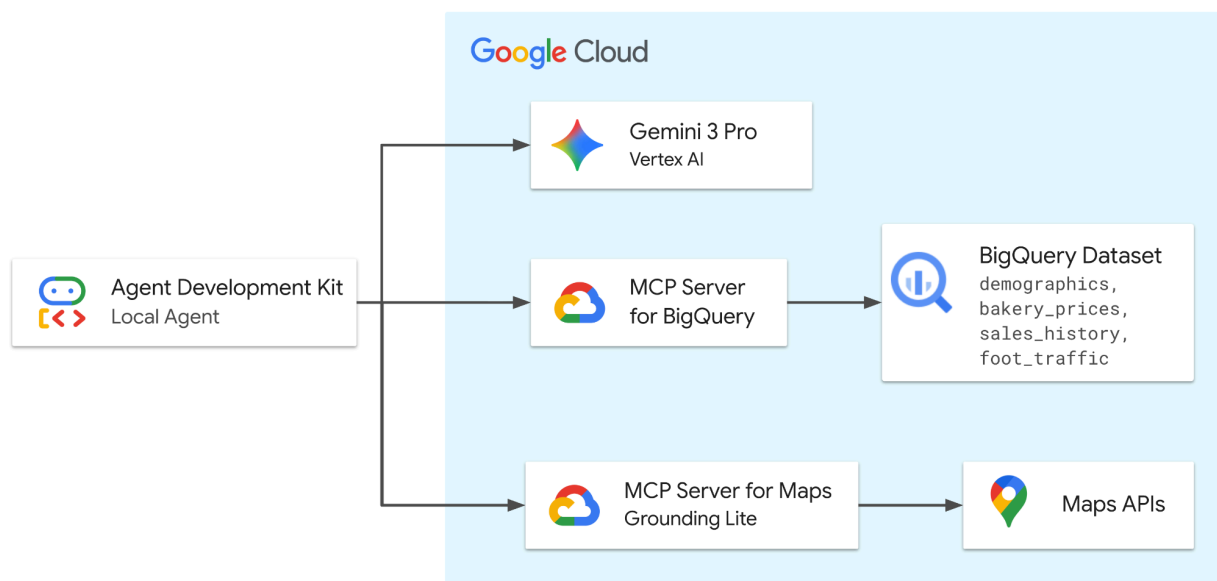
目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 取得程式碼](#)
4. [4. 設定環境和 BigQuery](#)
5. [5. 安裝 ADK](#)
6. [6. 檢查 ADK 應用程式](#)
7. [7. 與虛擬服務專員對話！](#)
8. [8. 清除](#)
9. [9. 恭喜](#)

1. 簡介

在本程式碼研究室中，您將使用 ADK 建構以 Gemini 3.1 Pro 為基礎的服務專員。代理程式會配備兩個遠端 (Google 代管) MCP 伺服器的工具，安全地存取 BigQuery 的受眾特徵、定價和銷售資料，以及 Google 地圖的實際位置分析和驗證。

代理程式會協調使用者和 Google Cloud 服務之間的請求，解決與虛構麵包店資料集相關的業務問題。



執行步驟

- 設定資料：在 BigQuery 中建立基礎烘焙資料集。
- 開發代理：使用 Agent Development Kit (ADK) 建構智慧型代理。
- 整合工具：透過 MCP 伺服器，為代理提供 BigQuery 和 Google 地圖功能。
- 分析市場：與代理互動，評估市場趨勢和飽和度。

軟硬體需求

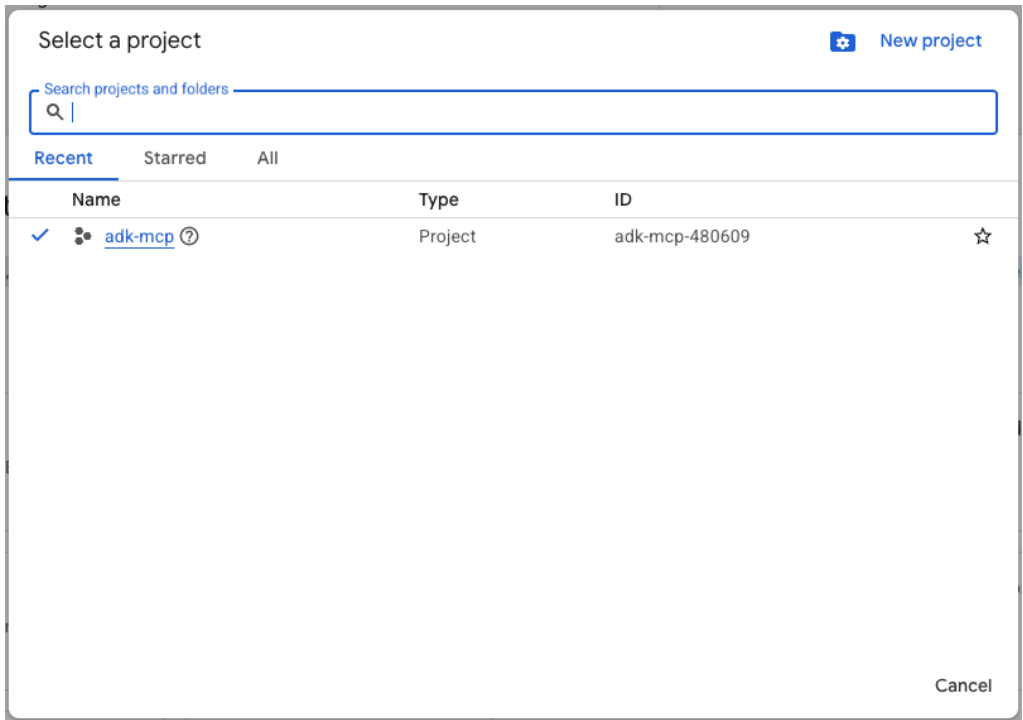
- 網路瀏覽器，例如 [Chrome](#)
- 已啟用計費功能的 Google Cloud 專案或 Gmail 帳戶。

本程式碼研究室適合各種程度的開發人員，包括初學者。您將使用 Google Cloud Shell 中的指令列介面和 Python 程式碼進行 ADK 開發。您不需要是 Python 專家，但瞭解如何解讀程式碼有助於理解概念。

2. 事前準備

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。

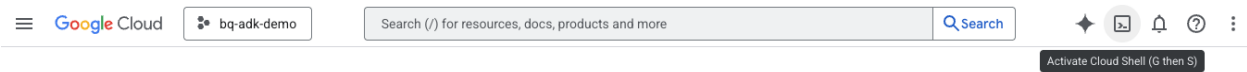


2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟動 Cloud Shell

Cloud Shell 是在 Google Cloud 中運作的指令列環境，已預先載入必要工具。

1. 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」：



2. 連線至 Cloud Shell 後，請執行下列指令，在 Cloud Shell 中[驗證您的驗證](#)：

〇〇〇〇

```
gcloud auth list
```

3. 執行下列指令，確認專案已設定為搭配 gcloud 使用：

〇〇〇〇

```
gcloud config get project
```

4. 確認專案符合預期，然後執行下列指令設定專案 ID：

〇〇〇〇

```
export PROJECT_ID=$(gcloud config get project)
```

步驟 3 · 約 5 分鐘

3. 取得程式碼

複製存放區

1. 將存放區複製到 Cloud Shell 殼層環境：

```
git clone https://github.com/google/mcp.git
```

2. 前往範例目錄：

```
cd mcp/examples/launchmybakery
```

驗證

執行下列指令，即可使用您的 Google Cloud 帳戶進行驗證。ADK 必須具備這項權限，才能存取 BigQuery。

```
gcloud auth application-default login
```

按照提示完成驗證程序。

附註：ADK 不會自動重新整理 OAuth 2.0 權杖。如果即時通訊工作階段超過 60 分鐘，您可能需要使用上述指令重新驗證。

4. 設定環境和 BigQuery

執行設定指令碼

1. 執行環境設定指令碼。這個指令碼會啟用 BigQuery 和 Google 地圖 API，並建立包含專案 ID 和 Maps API 金鑰的 `.env` 檔案。

```
chmod +x setup/setup_env.sh
./setup/setup_env.sh
```

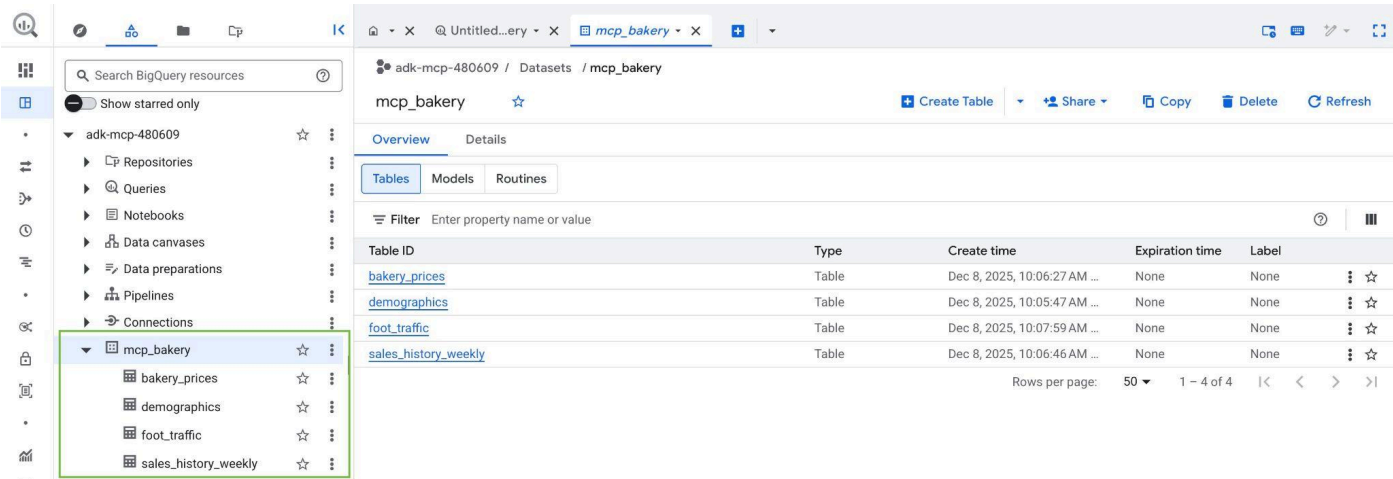
2. 執行 BigQuery 設定指令碼。這個指令碼會自動建立 Cloud Storage bucket、上傳資料，以及佈建 BigQuery 資料集和資料表。

```
chmod +x ./setup/setup_bigquery.sh
./setup/setup_bigquery.sh
```

指令碼執行完畢後，系統應會建立 `mcp_bakery` 資料集，並填入下列資料表：

- **受眾特徵**：郵遞區號的普查資料和人口特徵。
- **bakery_prices** - 各種烘焙產品的競爭對手價格和產品詳細資料。
- **sales_history_weekly** - 依商店和產品劃分的每週銷售業績 (數量和收益)。
- **foot_traffic** - 依郵遞區號和時段估算的來店人潮分數。

3. 前往 Google Cloud 專案中的 **BigQuery 控制台**，確認資料集和資料表已建立：



步驟 5 · 約 5 分鐘

5. 安裝 ADK

基礎架構已準備就緒，接下來請建立虛擬 Python 環境，並安裝 ADK 必要的套件。

1. 建立虛擬環境：

```
python3 -m venv .venv
```

2. 啟動虛擬環境：

```
source .venv/bin/activate
```

3. 安裝 ADK：

```
pip install google-adk==1.28.0
```

4. 前往代理程式目錄：

```
cd adk_agent/
```

步驟 6 · 約 5 分鐘

6. 檢查 ADK 應用程式

在 Cloud Shell 中按一下「Open Editor」(開啟編輯器) 按鈕，開啟 Cloud Shell 編輯器，並在 `mcp/examples/launchmybakery` 目錄下查看複製的存放區。



代理程式碼已在 `adk_agent/` 目錄中提供。讓我們來瞭解解決方案結構：

```
launchmybakery/  
├── data/                # Pre-generated CSV files for BigQuery  
├── adk_agent/           # AI Agent Application (ADK)  
│   └── mcp_bakery_app/  # App directory  
│       ├── agent.py     # Agent definition  
│       ├── tools.py     # Custom tools for the agent  
│       └── .env          # Project configuration (created by setup script)  
├── setup/               # Infrastructure setup scripts  
└── cleanup/             # Infrastructure cleanup scripts
```

「`mcp_bakery_app`」中的重要檔案：

- **`agent.py`**：定義代理程式、工具和模型 (Gemini 3.1 Pro 預先發布版) 的核心邏輯。
- **`tools.py`**：包含任何自訂工具定義。
- **`.env`**：包含專案設定和設定指令碼建立的密鑰 (例如 API 金鑰)。

1. MCP 工具組初始化：

現在，請在編輯器中開啟 `adk_agent/mcp_bakery_app/tools.py`，瞭解 MCP 工具集如何初始化。

如要讓代理程式與 BigQuery 和 Google 地圖通訊，我們需要設定 Model Context Protocol (MCP) 用戶端。

程式碼會使用 [StreamableHTTPConnectionParams](#)，與 Google 的遠端 MCP 伺服器建立安全連線。

```

def get_maps_mcp_toolset():
    dotenv.load_dotenv()
    maps_api_key = os.getenv('MAPS_API_KEY', 'no_api_found')

    tools = MCPToolset(
        connection_params=StreamableHTTPConnectionParams(
            url=MAPS_MCP_URL,
            headers={
                "X-Goog-API-Key": maps_api_key
            }
        )
    )
    print("MCP Toolset configured for Streamable HTTP connection.")
    return tools

def get_bigquery_mcp_toolset():

    credentials, project_id = google.auth.default(
        scopes=["https://www.googleapis.com/auth/bigquery"]
    )

    credentials.refresh(google.auth.transport.requests.Request())
    oauth_token = credentials.token

    HEADERS_WITH_OAUTH = {
        "Authorization": f"Bearer {oauth_token}",
        "x-goog-user-project": project_id
    }

    tools = MCPToolset(
        connection_params=StreamableHTTPConnectionParams(
            url=BIGQUERY_MCP_URL,
            headers=HEADERS_WITH_OAUTH
        )
    )
    print("MCP Toolset configured for Streamable HTTP connection.")
    return tools

```

- **地圖工具組**：使用 API 金鑰設定與 [Maps MCP 伺服器](#) 的連線。
- **BigQuery 工具組**：這個函式會設定與 [BigQuery MCP 伺服器](#) 的連線。這項工具會使用 google.auth 自動擷取 Cloud 憑證、產生 OAuth Bearer 權杖，並將權杖插入 Authorization 標頭。

2. 代理程式定義：

現在，在編輯器中開啟 `adk_agent/mcp_bakery_app/agent.py`，查看代理程式的定義方式。

`LlmAgent` 會使用 `gemini-3.1-pro-preview` 模型進行初始化。

```

maps_toolset = tools.get_maps_mcp_toolset()
bigquery_toolset = tools.get_bigquery_mcp_toolset()

root_agent = LlmAgent(
    model='gemini-3.1-pro-preview',
    name='root_agent',
    instruction=f"""
        Help the user answer questions by strategically combining insights from two
        sources:

        1. **BigQuery toolset:** Access demographic (inc. foot traffic index),
        product pricing, and historical sales data in the mcp_bakery dataset. Do not use any other
        dataset.

        Run all query jobs from project id: {project_id}.

        2. **Maps Toolset:** Use this for real-world location analysis, finding
        competition/places and calculating necessary travel routes.

        Include a hyperlink to an interactive map in your response where
        appropriate.
        """,
    tools=[maps_toolset, bigquery_toolset]
)

```

- **系統指令：**系統會提供具體指令，要求代理程式整合 BigQuery (資料) 和 Google 地圖 (位置分析) 的洞察資料。
- **工具：**系統會將 `maps_toolset` 和 `bigquery_toolset` 都指派給代理，讓代理存取這兩項服務的功能。

代理會遵守存放區中定義的指令和工具。歡迎變更指令，看看這對代理程式的行為有何影響。

步驟 7 · 約 10 分鐘

7. 與虛擬服務專員對話！

返回 Cloud Shell 中的終端機，執行下列指令前往 `adk_agent` 目錄 (如果尚未執行)：

```
cd adk_agent/
```

執行下列指令，啟動 ADK 網頁介面。這個指令會啟動輕量型網路伺服器，用來代管聊天應用程式：

```
adk web --allow_origins 'regex:https://.*\.cloudshell\.dev'
```

伺服器啟動後，Cloud Shell 會顯示下列內容：

```
+-----+
| ADK Web Server started                               |
|                                                       |
| For local testing, access at http://127.0.0.1:8000.  |
+-----+

INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

您可以透過兩種方式存取 ADK 使用者介面：

選項 1：按一下本機連結 按一下 Cloud Shell 終端機中顯示的 `http://127.0.0.1:8000` 連結。

選項 2：使用網頁預覽

1. 按一下 Cloud Shell 右上角的「網頁預覽」按鈕。
2. 選取「變更連接埠」。
3. 輸入通訊埠編號 **8000**，然後按一下「變更並預覽」。

```
(.venv) nelsonjeffrey@cloudshell:~/mcp/examples/launchmybakery$ adk web service
2026-03-02 03:48:56,892 - INFO - service_factory
2026-03-02 03:48:56,892 - INFO - local_storage.py
/home/nelsonjeffrey/mcp/examples/launchmybakery/adk_agent/.venv/lib/python3.12/site-packages/google/adk/auth/credential_service.py:192: UserWarning: [EXPERIMENTAL] InMemoryCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  credential_service = InMemoryCredentialService()
/home/nelsonjeffrey/mcp/examples/launchmybakery/.venv/lib/python3.12/site-packages/google/adk/auth/credential_service/in_memory_credential_service.py:33: UserWarning: [EXPERIMENTAL] BaseCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  super().__init__()
INFO: Started server process [3273]
INFO: Waiting for application startup.

-----
| ADK Web Server started
| For local testing, access at http://127.0.0.1:8000.
|-----
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

在網頁 UI 中提出下列問題，與 Agent 互動。您應該會看到系統呼叫相關工具。

- 1. 尋找鄰近地區 (巨集)：「我想在洛杉磯開一家烘焙坊，找出上午來店人潮分數最高的郵遞區號。」

Agent Development Kit

SESSION ID e162edd1-ee73-4067-8fe1-98c6d644b655 USER ID user

Token Streaming | + New Session

I want to open a bakery in Los Angeles. Find the zip code with the highest morning foot traffic score

Invocations

I want to open a bakery in Los Angeles. Find the zip code with

- list_table_ids
- list_table_ids
- get_table_info
- get_table_info
- get_table_info
- list_table_ids
- list_table_ids
- get_table_info
- get_table_info
- get_table_info
- get_table_info
- execute_sql
- execute_sql

The zip code in Los Angeles with the highest morning foot traffic score is 90403 (Santa Monica), with a perfect score of 100.0.

代理程式應使用 `get_table_info` 和 `execute_sql` 等工具，查詢 BigQuery 中的 `foot_traffic` 資料表。

- 2. 驗證位置 (微觀)：「您可以在該郵遞區號中搜尋『麵包店』，看看是否已飽和嗎？」

Can you search for 'Bakeries' in that zip code to see if it's saturated?



search_places

✓ search_places

Based on a search for "Bakeries in 90403," I found several established competitors in Santa Monica. It does appear to be a competitive market with some well-regarded options:

- **Petitgrain Boulangerie** (1209 Wilshire Blvd)
- **Paderia Bakehouse** (3032 Wilshire Blvd Suite E)
- **La Monarca Bakery & Cafe** (1300 Wilshire Blvd)
- **Sweet Lady Jane Santa Monica** (1631 Montana Ave)

While there are strong existing businesses, the "saturation" depends on your specific concept (e.g., are you doing donuts, artisanal bread, or cupcakes?) and whether you can offer something distinct from these existing options. The high foot traffic score (100.0) suggests there is plenty of demand to go around, but you'll be sharing the street with these neighbors.

You can explore the area and these locations on this [interactive map](#).

代理程式應使用地圖工具集中的 `search_places` 工具回答這個問題。

快來試試看！請參閱下列範例問題，瞭解 ADK 代理的實際運作情形：

- 「我打算在洛杉磯開設第四家麵包店，我需要早起活動的社區。找出『上午』來店人潮分數最高的郵遞區號。」
- 「請在該郵遞區號中搜尋『麵包店』，看看是否已飽和。如果太多，請檢查『特選咖啡』商店，這樣我才能在附近找個位置，吸引來店人潮。」
- 「好的，我想將這個品牌定位為頂級品牌。在洛杉磯都會區，『酸麵包』的最高價格是多少？」
- 「現在我想要 2025 年 12 月的收益預測。查看我的銷售記錄，並從銷售業績最佳的商店取得「酸種麵包」的資料。執行 2025 年 12 月的預測，估算我將售出的數量。然後，使用我們找到的優質價格 (假設為 \$18) 計算預估總收益」
- 「這樣就夠付房租了。最後，請確認物流資訊。找出建議區域附近最近的「Restaurant Depot」，並確認每日補貨的車程時間在 30 分鐘內。」

測試完代理後，只要在 Cloud Shell 終端機按下 `Ctrl+C`，即可終止 ADK 網頁介面。

步驟 8 · 約 0 分鐘

8. 清除

如要避免系統持續向您的 Google Cloud 帳戶收費，請刪除在本程式碼研究室中建立的資源。

執行清除指令碼。這個指令碼會刪除設定期間建立的 BigQuery 資料集、Cloud Storage bucket 和 API 金鑰。

```
chmod +x ../cleanup/cleanup_env.sh
../cleanup/cleanup_env.sh
```

9. 恭喜

任務完成！ 您已使用 Agent Development Kit (ADK) 成功建構位置情報代理。

您已在 **BigQuery** 中的「企業」資料與 **Google 地圖** 的實際位置脈絡之間建立橋樑，打造出強大的工具，能夠進行複雜的業務推論，一切都由 Model Context Protocol (MCP) 和 Gemini 驅動。

完成的目標：

- **基礎架構即程式碼**：您使用 Google Cloud CLI 工具佈建資料堆疊。
- **MCP 整合**：您將 AI 代理連線至兩個不同的遠端 MCP 伺服器 (BigQuery 和 Maps)，不必編寫複雜的 API 包裝函式。
- **統一推論**：您建構的單一代理程式能夠策略性地結合兩個不同領域的洞察資料，解決業務問題。

參考文件

- [使用 BigQuery MCP 伺服器](#)
 - [使用 Maps Grounding Lite MCP 伺服器](#)
 - [驗證 MCP](#)
-